

Sparse Exploratory Approximate Factor Analysis: A Tutorial for the R package seafar

Tra Le¹, Stefan Kirsch², and Katrijn Van Deun¹

¹Department of Methodology and Statistics, Tilburg University

²Research Support Team, Tilburg University

Author Note

Tra Le  <https://orcid.org/0000-0002-7700-8946>

Stefan Kirsch  <https://orcid.org/0009-0009-6939-411X>

Katrijn Van Deun  <https://orcid.org/0000-0002-2271-793X>

The authors have no conflicts of interest to declare that are relevant to the content of this article. This publication is part of the project SEM2.0 (with project number NWO 406.22.GO.022) of the Open Competition research program which is financed by the Dutch Research Council (NWO).

Correspondence concerning this article should be addressed to Tra Le, Email: t.t.le_1@tilburguniversity.edu

Abstract

This paper provides a tutorial for the R package *seafar*, which implements Sparse Exploratory Approximate Factor Analysis. The package is designed for exploratory research settings with many variables, limited sample sizes, and little prior knowledge about the underlying factor structure, where traditional latent variable methods may perform poorly or become unstable. The package has three main aims: (i) to provide users with easy control over the exact number of nonzero/zero loadings for the simple structure of the factor loadings, (ii) to provide data-driven model selection procedures for determining the number of (non)zero loadings in exploratory settings, and (iii) to estimate factor scores which can be used in further analyses. The use of the *seafar* package is demonstrated using two empirical data sets.

Keywords: exploratory factor analysis, high-dimensional data, tutorial, R package

Sparse Exploratory Approximate Factor Analysis: A Tutorial for the R package *seafar*

The increasing availability of large-scale data and technologically advanced measurement tools has led to data sets containing more variables than observations, often referred to as low-sample-size high-dimensional data. In behavioral research, such data frequently motivate questions about the latent constructs underlying the observed variables. A natural framework for addressing these questions is exploratory factor analysis (EFA), which seeks to identify latent constructs that account for the covariance structure among observed variables ([Morin, 2014](#)). Traditional EFA methods, however, may perform poorly in complex settings characterized by many unknown parameters relative to the sample size, particularly when little prior theoretical information is available and/or the sample size is limited. As a result, there has been growing interest in regularization-based methods that enable estimation in such situations. Examples include Penalized Factor Analysis in R (`penfa`, [Geminiani et al., 2021b, 2021a](#)), Penalized Structural Equation Modeling in Mplus ([Asparouhov & Muthén, 2024](#)), and Penalized Likelihood SEM in R (`lsix`, [Huang et al., 2017](#); [Huang, 2020](#)), among others.

Le et al. ([2026](#)) recently proposed Regularized ESEM with the same general aim of conducting exploratory factor analysis for complex models and high-dimensional data. the method showed favorable performance relative to competing regularized approaches in recovering the simple structure of the loading matrix (i.e., correctly recovering which variables load on the factor and efficient estimation of loadings), while also demonstrating stronger convergence behavior. However, the method was previously available only as a collection of functions in R hosted on GitHub, which may limit routine use for applied researchers. To support users who want to conduct a Regularized ESEM analysis, the present paper offers a tutorial on the use of the *seafar* package, which implements (part of) Regularized ESEM with ready-to-use functions with a few modifications and extensions. First, whereas the original Regularized ESEM used separate regularization approaches for orthogonal and non-orthogonal factors (i.e., cardinality constraint and the LASSO penalty), *seafar* provides a unified framework based on cardinality constraints for both settings. This choice was motivated by the ability of the cardinality-constrained approach to

result in exactly the number of (non)zero loadings prespecified by the users. This is not the case when using the LASSO penalty. Second, *seafar* includes the original model selection procedure based on the Index of Sparseness, as well as an alternative warm-start strategy that can reduce computational time. The latter is especially helpful when the data set contains thousands of variables.

This tutorial is intended to help researchers apply the *seafar* package to perform a sparse exploratory factor analysis in practice. Accordingly, we first introduce the Sparse Exploratory Approximate Factor Analysis (SEAFa) model as part of the original Regularized ESEM and its model selection procedures. Next, we demonstrate the use of *seafar* with two empirical data sets. Finally, we conclude with practical recommendations and remarks for applied researchers.

Method

In this section, we will introduce the notation used in this paper, the SEAFa model, its estimation, and model selection procedures.

Data and notation

Here, we introduce the notation used throughout this paper. Matrices are denoted by bold uppercase letters and vectors by bold lowercase letters. Transposes are indicated by the superscript T , and scalars are denoted by lowercase italic letters. For a matrix $\mathbf{A}$, $\text{Card}(\mathbf{A})$ denotes the number of nonzero elements in $\mathbf{A}$. $\mathcal{T}_C(\mathbf{A})$ indicates the matrix obtained from $\mathbf{A}$ by retaining the C entries with the largest absolute values and setting all remaining entries to zero.

Let $\mathbf{Y} \in \mathbb{R}^{N \times J}$ be the data matrix of interest consisting of J standardized items (i.e., items are mean-centered and scaled to unit variance) for N observations. The squared Frobenius norm of $\mathbf{Y}$ is denoted as $\|\mathbf{Y}\|_2^2 = \sum_{i=1}^N \sum_{j=1}^J y_{ij}^2$.

The SEAFa model

It is important to point out that the SEAFa model is essentially the measurement model of Regularized ESEM (Le et al., 2026). Here, we outline the details of the model again. Let $\mathbf{y}$ be a $J \times 1$ vector of standardized scores on the J observed variables, $\mathbf{P}$ be a $J \times Q$ factor loading

matrix, and $\boldsymbol{\eta}$ be a $Q \times 1$ vector of factor scores, we have the SEAFa model as follows:

$$\mathbf{y} = \mathbf{P}\boldsymbol{\eta} + \boldsymbol{\epsilon} \quad (1)$$

where $\boldsymbol{\epsilon}$ is the $J \times 1$ vector of residuals. Model (1) is subject to two identification constraints: 1) $\mathbf{P}$ is assumed to show simple structure (i.e., not all items load on all factors but some/many have zero loadings), and 2) factor scores $\boldsymbol{\eta}$ are subject to norm restriction (i.e., sum of squared factor scores is equal to N). Furthermore, the residuals $\boldsymbol{\epsilon}$ are assumed to have mean zero and to be uncorrelated with the factor scores. Here, residuals are allowed to be weakly correlated, in contrast to the common factor model where residuals are assumed to be uncorrelated.

As discussed in Le et al. (2026), a least-squares approach is used to estimate the approximate factor model in (1), as it has favorable properties in high-dimensional settings (Fan et al., 2021). Let p_{jq} denote the loading of variable j on factor q and η_{iq} denote the score of person i on factor q , the least-squares objective function is as follows:

$$\begin{aligned} & \arg \min_{\boldsymbol{\eta}, \mathbf{P}} \sum_{j=1}^J \sum_{i=1}^N (y_{ij} - \sum_q \eta_{iq} p_{jq})^2 \\ & \text{subject to } \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q, \end{aligned} \quad (2)$$

and $\mathbf{P}$ showing simple structure.

Regularized ESEM (Le et al., 2026) chose to impose simple structure on $\mathbf{P}$ in two ways: i) putting a hard constraint on $\mathbf{P}$ to have exactly C nonzero loadings (for orthogonal factors); ii) using the LASSO penalty to shrink (many) loadings toward zeros (for non-orthogonal factors). Here, SEAFa unifies these approaches by enforcing simple structure solely through a cardinality constraint on $\mathbf{P}$, for both orthogonal and non-orthogonal factors. Thus, the objective function of SEAFa is as follows:

$$\begin{aligned} & \arg \min_{\boldsymbol{\eta}, \mathbf{P}} \sum_{j=1}^J \sum_{i=1}^N (y_{ij} - \sum_q \eta_{iq} p_{jq})^2 \\ & \text{subject to } \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q, \end{aligned} \quad (3)$$

and $\text{Card}(\mathbf{P}) = C$,

or in matrix form

$$\begin{aligned} & \arg \min_{\mathbf{H}, \mathbf{P}} \|\mathbf{Y} - \mathbf{H}\mathbf{P}^T\|_F^2, \\ & \text{subject to } \boldsymbol{\eta}_q^T \boldsymbol{\eta}_q = N \quad \forall q = 1, \dots, Q, \\ & \text{and } \text{Card}(\mathbf{P}) = C. \end{aligned} \quad (4)$$

To solve (4), we use an alternating optimization (AO) scheme: factor scores $\mathbf{H}$ and loadings $\mathbf{P}$ are updated alternately while holding the other fixed. Specifically, with fixed $\mathbf{P}$, the estimate of the factor scores $\hat{\mathbf{H}}$ (subject to norm constraint) is obtained using the method of Lagrange multipliers. $\hat{\mathbf{P}}$ is estimated with fixed $\mathbf{H}$ via the cardinality-constrained linear regression problem:

$$\begin{aligned} \hat{\mathbf{P}} &= \arg \min_{\mathbf{P}} \|\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H}) \text{vec}(\mathbf{P}^T)\|_2^2, \\ & \text{subject to } \text{Card}(\mathbf{P}) = C, \end{aligned} \quad (5)$$

where $\otimes$ is the Kronecker product and $\text{vec}(\cdot)$ stacks the columns of a matrix into a single column vector. $\hat{\mathbf{P}}$ is updated using a numerical procedure proposed by Adachi and Kiers (2017) as:

$$\mathbf{P}_{\text{new}} = \mathcal{T}_C(\mathbf{B}), \quad (6)$$

where $\mathbf{B} = \mathbf{P}_{\text{old}} - \alpha^{-1}(\mathbf{P}_{\text{old}}\mathbf{H}^T\mathbf{H} - \mathbf{Y}^T\mathbf{H})$ with α being the largest eigenvalue of $\mathbf{H}^T\mathbf{H}$. The detailed derivation of $\mathbf{B}$ can be found in Appendix A. This update can be seen as a solution to a projected gradient scheme with fixed step size α^{-1} (Guerra-Urzola et al., 2023). In the special case of orthogonal factors, that is, when $\mathbf{H}^T\mathbf{H} = N\mathbf{I}$, Adachi and Trendafilov (2016) proposed a computationally efficient update of $\mathbf{H}$ and $\mathbf{P}$ as follows: given $\mathbf{P}$, $\hat{\mathbf{H}}$ is the solution to the Orthogonal Procrustes problem; given $\mathbf{H}$, $\hat{\mathbf{P}}$ is updated using (6), but with $\mathbf{B} = N^{-1}\mathbf{Y}^T\mathbf{H}$. In both cases, the cardinality constraint may also be imposed separately for each factor, such that $\text{Card}(\mathbf{p}_q) = C_q$. This yields a factor-wise update of the loadings:

$$\mathbf{p}_q = \mathcal{T}_{C_q}(\mathbf{b}_q), \quad q = 1, \dots, Q.$$

The AO procedure results in a non-increasing sequence of loss values. However, as (4) is not a convex problem, the algorithm is subject to local minima. Thus, we implement the algorithm with

a multi-start procedure: the algorithm is run several times with different starting values and the solution with the lowest value is retained as the final solution. The implementation of SEAFa is summarized in Algorithm 1.

Model selection

Algorithm 1 requires five inputs: a matrix of standardized observed variables $\mathbf{Y}$, number of factors Q , number of nonzero loadings C , and two stopping criteria. While all inputs can be user-specified, the number of factors and nonzero loadings may be unknown beforehand. Thus, we propose a sequential data-driven strategy similar to Le et al. (2026) to choose these two inputs:

- Step 1: Number of factors Q can be specified either based on prior knowledge or by using different factor retention methods such as parallel analysis (Horn, 1965), scree plot (Cattell, 1966), Kaiser-Guttman rule (Kaiser, 1960), or a machine learning method (Goretzko & Bühner, 2020).
- Step 2: Based on the selected number of factors, the number of nonzero loadings C is chosen using the Index of Sparseness (Gajjar et al., 2017; Trendafilov, 2014). That is, the algorithm searches through a sequence of candidate values for C and chooses the one that maximizes the Index of Sparseness (IS), which is defined as follows:

$$IS = PEV_{PCA} \times PEV_{rESEM} \times PS, \quad (7)$$

where PEV_{PCA} , PEV_{SEAFa} and PS are the proportion of explained variance using ordinary PCA, the proportion of explained variance using SEAFa, and the proportion of zero loadings, respectively. A suitable range of C is $Q \times 3$ (at least three nonzero loading per factor) to $J \times Q$.

Since the IS has a preference for parsimonious models at the cost of fit, we used a modified version of the IS to compensate for this tendency. The details of the modified IS can be found at Van Deun et al. (2025) and its implementation in the *seafar* package will be explained in the later section (Section Analysis with unknown cardinality). Furthermore, in Step 2, PEV_{SEAFa} is

Algorithm 1: Sparse Exploratory Approximate Factor Analysis (SEAFA)

Input: $\mathbf{Y}, Q, C$, maximum number of iterations *maxiter*, convergence threshold ε **Output:** $\hat{\mathbf{P}}, \hat{\mathbf{H}}$ Initialize loading matrix $\mathbf{P}^{(0)}$ having C nonzero loadings**while** $\Delta lossfunction > \varepsilon$ *or* $iteration \neq maxiter$ **do** **if** η_q are orthogonal **then** $\mathbf{H} \leftarrow \sqrt{N}\mathbf{U}\mathbf{V}$ from the SVD of $N^{-1/2}\mathbf{Y}\mathbf{P} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ $\mathbf{B} \leftarrow N^{-1}\mathbf{Y}^T\mathbf{H}$ **else** **while** $\Delta lossfunction2 > \varepsilon$ *or* $iteration \neq maxiter$ **do** **for** $q \leftarrow 1$ to Q **do** $\mathbf{E}_q \leftarrow \mathbf{Y} - \mathbf{H}\mathbf{P}^T + \eta_q \mathbf{p}_q^T$ $\eta_q \leftarrow \frac{\sqrt{N}\mathbf{E}_q \mathbf{p}_q}{\sqrt{(\mathbf{E}_q \mathbf{p}_q)^T (\mathbf{E}_q \mathbf{p}_q)}}$ **end** **end** $\alpha \leftarrow$ maximum eigenvalue of $\mathbf{H}^T \mathbf{H}$ $\mathbf{B} = (\mathbf{P}^T - \frac{\mathbf{H}^T \mathbf{H} \mathbf{P}^T - \mathbf{H}^T \mathbf{Y}}{\alpha})^T$ **end** **if** cardinality per factor **then** **for** $q \leftarrow 1$ to Q **do** $\mathbf{p}_q \leftarrow T_{C_q}(\mathbf{b}_q)$ **end** **else** $\mathbf{P} \leftarrow \mathcal{T}_C(\mathbf{B})$ **end****end****return** $\hat{\mathbf{P}}, \hat{\mathbf{H}}$

obtained by fitting SEAFA for each candidate value of C using multiple starting values. This procedure can become computationally intensive when the number of variables J is large, as the candidate range for C increases accordingly. To mitigate this issue, we propose an additional strategy based on warm starts (Friedman et al., 2010). Specifically, for a decreasing sequence of candidate values of C , the loading matrix obtained for a given C is used as the starting value for the subsequent (smaller) value of C , while computing the IS at each step. This can help speed up the computational time substantially; however, the results are more dependent on the initial solution, as subsequent estimates build on the loading matrix obtained in the first iteration.

The seafar package

In this section, we demonstrate the use of our package in R using two empirical datasets. The first analysis focuses on recovering the loading patterns of the Big Five dataset (low-dimensional scenario), while the second analysis explores the factor scores of the Autism Genetic dataset from Nishimura et al. (2007) (ultra-high-dimensional scenario). Our package requires the *PCAtools* package which was not available on CRAN at the time of writing. This can lead to problems when installing *seafar* from GitHub. We therefore recommend installing *PCAtools* from *BiocManager* before installing *seafar* as follows:

```
install.packages("BiocManager")
BiocManager::install("PCAtools")

install.packages("remotes")
remotes::install_github("trale97/seafar")
library(seafar)
```

Analysis 1: Factor loading structure of the Big Five

Data pre-processing

The Big Five dataset can be obtained directly from the *qgraph* package in R (Epskamp et al., 2012). The dataset contains scores of 500 observations on the 240 items of the personality questionnaire. All functions in *seafar* work with standardized items, thus, each variable is standardized to have a mean of zero and unit variance before the analysis:

```
data("big5", package = "qgraph")
big5_std <- scale(big5, center = TRUE, scale = TRUE)
```

Number of factors

In line with the design of the questionnaire used to collect the data, we assume five factors in correspondence with the five personality traits. Alternatively, one can use different data-driven methods to extract number of factors by using the `factor_retention()` function in the *seafar* package. This function calls on existing R implementations of the three factor retention methods, namely parallel analysis from package *psych* (William Revelle, 2026) and *PCAtools* (Blighe et al., 2026), scree plot from *PCAtools*, and Kaiser-Guttman rule from *EFA.dimensions* (O'Connor, 2026).. The results can be inspected by calling `summary()` as below:

```
library(seafar)
Q <- factor_retention(data = big5_std)
summary(Q)
```

Number of factors by parallel analysis is: 18

Number of factors by Kaiser rule is: 12

Number of components by scree test is: 5

Analysis with known cardinality

Given the design of the questionnaire (with each of the items constructed to measure one trait), we first focus on a solution with 240 nonzero loadings and the remaining 960 zero. We perform a SEAFA analysis using a preset cardinality of 240 with a multi-start procedure:

```
big5_multistart <- seafar_multistart(
  data = big5_std,
  nfactors = 5,
  C = 240,
  INIT = "rational"
)
```

Note that here, we did not restrict the five factors to be orthogonal. Users can specify `orthogonal = TRUE` to impose orthogonality on the factors. The default number of starting values is 50, which can be modified by option `nstarts`. Users can also specify the number of nonzero loadings *per* factor instead of across all factors. Since the questionnaire was designed to measure each trait with 48 items, we indicate a cardinality of 48 nonzero loadings per factor by changing the input `C = rep(48, 5)`. Furthermore, users can choose the method to initialize the loading matrix by specifying the option `INIT` with the following choices:

- `INIT = rational`: the initialization is inspired by the approach by Ding and He (2004) where principal components can be interpreted as continuous relaxations of cluster membership indicators. Here, for the initialization of $\mathbf{P}$, an SVD-based (Singular Value Decomposition) loading matrix is first computed, which are then used to cluster the variables into Q clusters. The resulting cluster structure is then used as a target for rotation, yielding a loading matrix with a simple structure with no cross-loadings. This choice is motivated by the common assumption in questionnaire data that items primarily load on a single latent factor with no cross-loadings (Reise et al., 2000). The resulting rotated loading matrix is used as the initial loading matrix for SEAFA.
- `INIT = random`: the initial loading matrix have complete random values sampled from a multivariate normal distribution $MVN(\mathbf{0}, \mathbf{I})$.
- `INIT = semirational`: a semi-rational initialization in which the initial loading matrix is constructed as a weighted combination of an SVD-based start (80%) and a random start (20%). The chosen weights emphasize the stability of the SVD-based start while introducing a modest amount of randomness. This is the default option when nothing is specified.

The results can be summarized using `summary()` function with option `disp =` with two values:

- `disp = "loadings"` only prints the first ten rows of the loading matrix.

- `disp = "full"` prints both the factor score and loading matrices.

```
summary(big5_multistart, disp = "loadings")
```

The number of nonzero loadings is: 240

The estimated loadings matrix is

	[,1]	[,2]	[,3]	[,4]	[,5]
N1	0	0	0	0.569	0
E2	0	0.366	0	0	0
O3	0	0	0.592	0	0
A4	0	0.462	0	-0.313	0
C5	0	0	0	0	0.464
N6	0	0	0	0.478	0
E7	0.453	0	0	0	0
O8	0	0	0.536	0	0
A9	0	0.503	0	0	0
C10	0	0	0	0	0.238

The full factor score and loading matrices can be extracted by calling `big5_multistart$scores` and `big5_multistart$loadings`, respectively. This is useful when users wish to use the full factor scores and/or loadings for further analyses, such as interpreting the factors or examining correlations among them. As can be seen, with the rational starting values for the loading matrix, each item only loads on one factor and there is no cross-loading. Table 1 shows the number of items loading on the factor, with items categorized into the five personality traits they were designed for. As could be expected, the large majority of items measuring a particular personality trait load on the same factor (note that this is an exploratory analysis that does not incorporate prior information about the questionnaire design, aside from the number of factors.).

Table 1*The number of nonzero loadings per factor corresponding to each personality trait*

Traint	Factor 1	Factor 2	Factor 3	Factor 4	Factor 5
Openness	30	2	0	2	1
Concientiousness	0	42	6	2	5
Extraversion	3	1	33	10	6
Agreeableness	2	0	7	35	2
Neuroticism	1	3	5	4	38

Note. The factors were ordered such that the largest number of nonzero loadings is on the diagonal.

The above analyses were done using a multi-start procedure. The package also offers a single-start procedure by using the `seafar()` function as follows:

```
big5_result <- seafar(
  data = big5_std,
  nfactors = 5,
  C = 240,
  INIT = "svd"
)
```

Here, the `INIT` input offers an extra option for starting values in addition to the starting configurations above:

- `INIT = svd`: for starting values based on the Singular Value Decomposition of the items.

The single-start procedure is computationally faster than the multi-start one, especially when the items matrix is large. However, it does not guarantee to work well in all scenarios but rather is heavily dependent on the input data itself. Users are advised to use it with caution.

Analysis with unknown cardinality

Often in an exploratory setting, the number of nonzero loadings is not known beforehand. When this is the case, users can use the `is.seafar()` function for model selection based on the modified Index of Sparseness ([Van Deun et al., 2025](#)). The function requires similar inputs as the `seafar()` function with two additional ones: `TOL` and `THR`. First, 100 candidate solutions are inspected in increasing order of cardinality. In each solution, the smallest nonzero loading is compared to the user-defined threshold `THR`. If it is smaller than `THR`, the previous cardinality is taken as a preliminary upper bound. Next, among all candidate solutions with approximately equal PEV to this preliminary solution, the final cardinality is chosen as the one with the highest IS value. Approximate equality of PEV is determined by rounding the PEV values to `TOL` decimal places. In the example below, we use `THR = 0.2` and `TOL = 3`, corresponding to a minimum loading of 0.2 and comparison of PEV up to three decimal points.

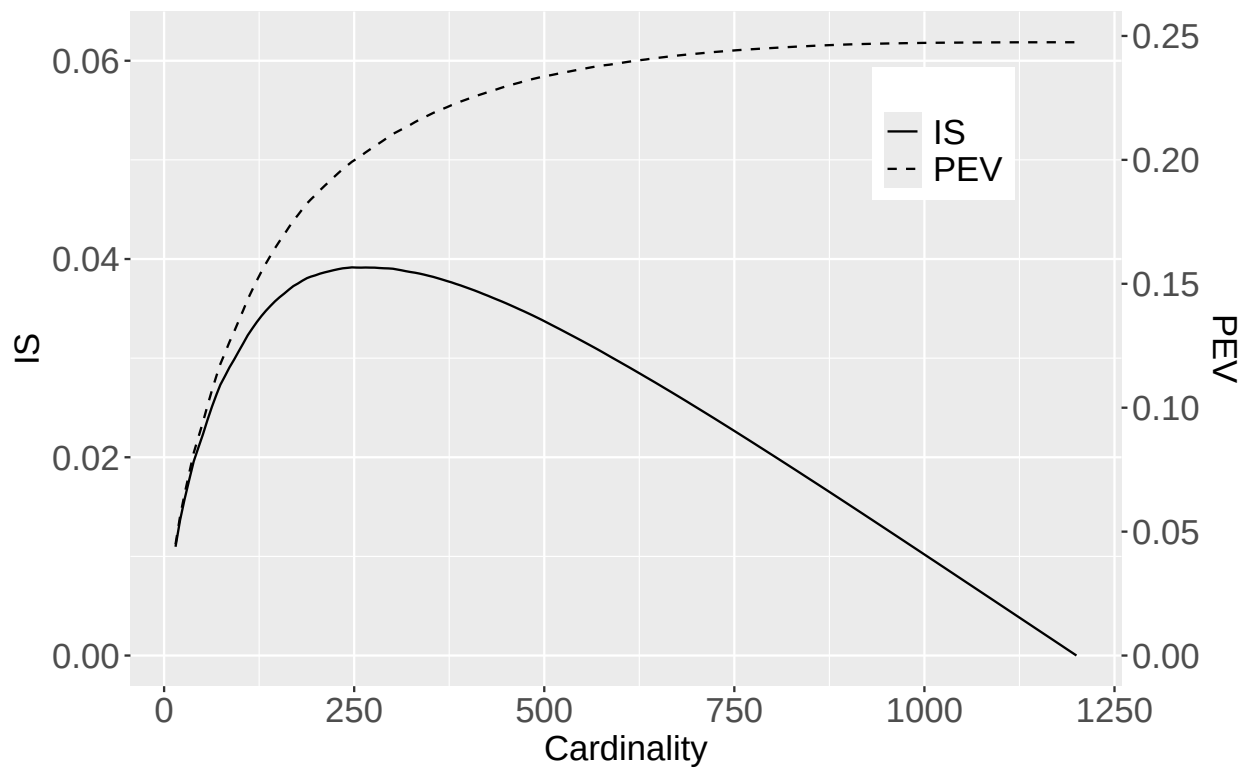
```
model_selection <- is.seafar(  
  data = big5_std,  
  nfactors = 5,  
  INIT = "rational",  
  nstarts = 10,  
  TOL = 3,  
  THR = .2  
)
```

When the dataset is large, this model selection procedure can be computationally intensive. Thus, users can use the `is.seafar_wst()` function that makes use of warm-starts as implemented by the *glmnet* package ([Friedman et al., 2010](#)) to speed up the computation time. This function also allows users to specify the number of candidate values to be tested within this range; here, we chose 200 values.

```
model_selection_wst <- is.seafar_wst(  
  data = big5_std,  
  nfactors = 5,  
  INIT = "rational",  
  card.length = 200
```

Figure 1

Index of sparseness (IS) and proportion of explained variance (PEV) against cardinality level for the Big Five data set



```
)
summary(model_selection_wst)
```

The cardinality selected by the Index of Sparseness is: 247

The PEV with this cardinality is: 0.199

The PEV of the unconstrained model (no zero loadings) is: 0.247

The `summary(model_selection_wst)` function displays the chosen cardinality and the proportion of explained variance (PEV) of the data associated with this level of cardinality, as well as the PEV of the unconstrained model (i.e., no zero loading). Figure 1 shows the evolution of PEV against the cardinality levels.

Analysis 2: Analysis of factor scores in Autism genetic data

As the *seafar* package was developed to specifically deal with high-dimensional setting, we also demonstrate how it can be used to analyze the Autism Genetic data set (Nishimura et al., 2007). The data is publicly available at <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE7329>. The data was preprocessed by removing three individuals who were mislabeled and standardizing each column of the data to have a mean of zero and a variance of one (as done in Guerra-Urzola et al., 2023)¹. The resulting data set contains 43893 genetic marker variables for 27 subjects, in which 14 individuals are in the control group, six are in the FMR1-FM autism group, and seven are in the dup15q autism group. This final data set can be load directly from the *seafar* package as follows:

```
library(seafar)
data("autism")
```

We chose three factors as indicated in the original paper by Nishimura et al. (2007). Due to the size of the data set, we perform a model selection with warm-start to determine the number of nonzero loadings from 200 candidate values:

```
autism_cardinality <- is.seafar_wst(
  data = autism,
  nfactors = 3,
  INIT = "rational",
  orthogonal = TRUE,
  card.length = 200
)

selcard <- autism_cardinality$selcard
summary(autism_cardinality)
```

The cardinality selected by the Index of Sparseness is: 37062

The PEV with this cardinality is: 0.251

¹ The detailed steps to retrieve the data from source and preprocess it can be found at

<https://github.com/trale97/seafar/blob/master/data/README.md>

The PEV of the unconstrained model (no zero loadings) is: 0.321

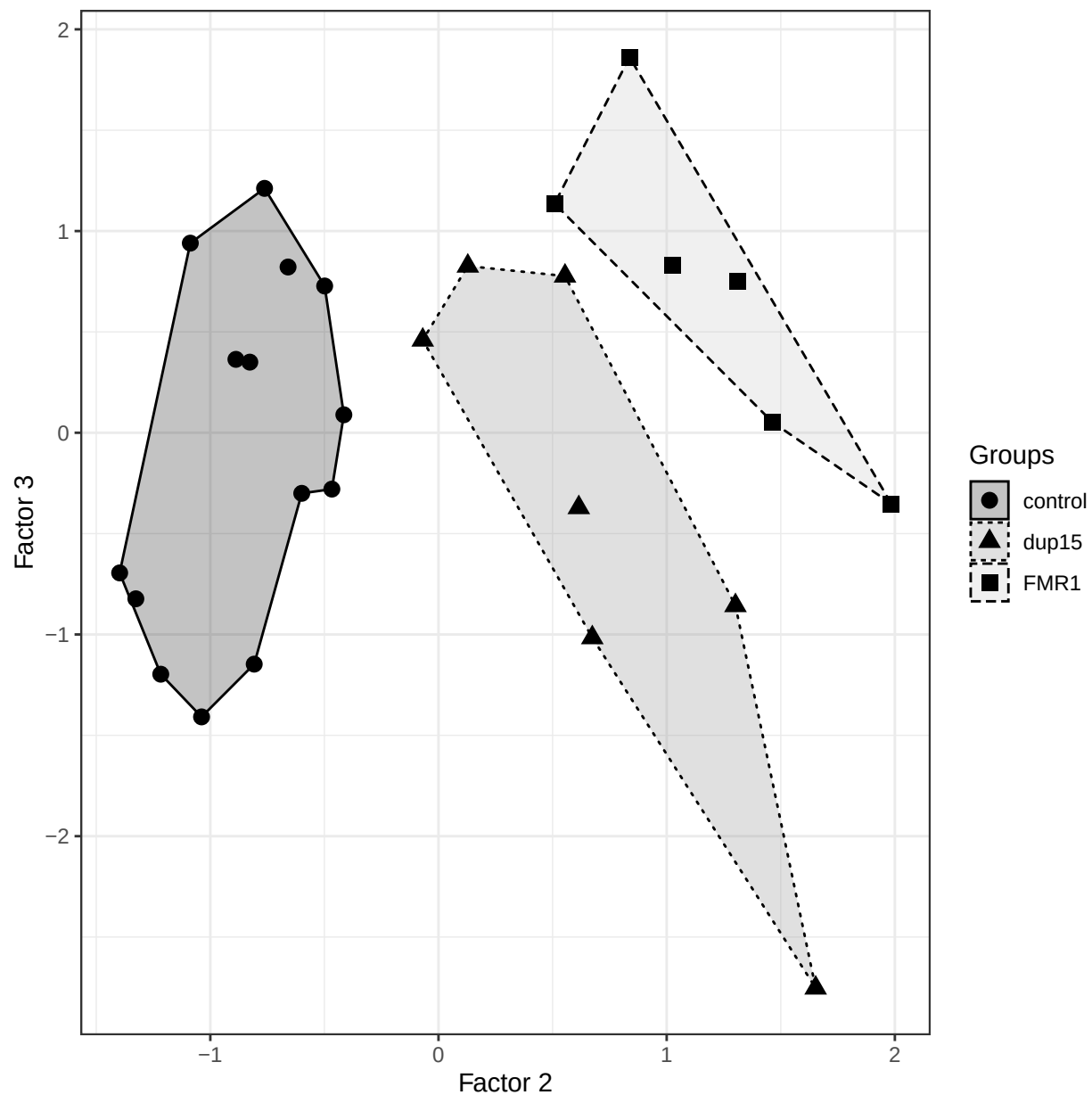
As can be seen, the chosen cardinality is 37062 nonzero loadings, which is less than a third of the total number of loadings (i.e., 43893×3 number of loadings). Using this value of cardinality, we run a SEAFa analysis with multiple starting values

```
autism_multistart <- seafar_multistart(  
  data = autism,  
  nfactors = 3,  
  C = selcard,  
  orthogonal = TRUE,  
  nstarts = 25  
)
```

In this analysis, the factor scores were extracted for subsequent analysis, by using command `autism_multistart$scores`. Figure 2 shows a plot of Factor 2 and Factor 3 plotted against each other. As can be seen, Factor 2 clearly separated the control group and the two autism groups. This reflects the same results by Nishimura et al. (2007), without having to use a filtering step (i.e., Nishimura et al. (2007) conducted an ANOVA to select a subset of 293 genetic markers that maximally discriminated between the three groups, which were then used in a PCA).

Discussion and Conclusions

In this tutorial, we demonstrated how to perform exploratory factor analysis using the R package *seafar* in settings with a large number of observed variables (e.g., the Autism data set) as well as in settings with many unknown parameters to be estimated relative to the sample size (e.g., the Big Five data), with little to no prior information on the loading structure and factors relationship. These are scenarios where *seafar* is most useful, while existing (regularized) EFA methods may struggle (Le et al., 2026). The underlying algorithm of *seafar* builds on the approximate factor model and extends the recently proposed Regularized ESEM approach (Le et al., 2026). In particular, *seafar* employs a cardinality constraint to impose simple structure on the loading matrix for both orthogonal and non-orthogonal factors. This constraint offers several advantages. First, it yields exact zero loadings, in contrast to rotation-based methods that rely on

Figure 2*Factor scores plot*

post-hoc thresholding of small but nonzero values. Second, it provides direct control over the number of nonzero loadings, whereas with the LASSO penalty, for instance, users do not know beforehand how many (non)zero loadings are associated with a particular level of penalty.

A limitation of the cardinality-constrained approach implemented in seafar arises in settings with non-orthogonal factors and a very large number of observed variables. Since the

update of the loadings involves sorting the entire loading matrix to keep the K elements with the largest absolute values, the computational time increases with the number of observed variables. Consequently, the LASSO-based approach proposed by Le et al. (2026) still offers an attractive alternative in settings with non-orthogonal factors and many items. This method is also implemented as a separate function in *seafar* but is not discussed in detail here. Interested readers are referred to Le et al. (2026) for the methodological derivation. An example illustrating the use of the LASSO approach in *seafar* is provided in [Appendix B](#).

The *seafar* package provides a user-friendly implementation of sparse exploratory factor analysis with minimal input requirements, along with built-in model selection procedures based on the Index of Sparseness. These procedures, suggested by RegularizedSCA (Gu et al., 2019) and Regularized ESEM (Le et al., 2026), allow users to determine the appropriate number of (non)zero loadings in the absence of strong prior theoretical guidance. The proposed package can be extended in several ways, for example to accommodate categorical variables, further broadening its applicability.

References

- Adachi, K., & Kiers, H. A. (2017). *Sparse regression without using a penalty function*.
http://www.jfssa.jp/taikai/2017/table/program_detail/pdf/1-50/10009.pdf
- Adachi, K., & Trendafilov, N. T. (2016). Sparse principal component analysis subject to prespecified cardinality of loadings. *Computational Statistics*, 31, 1403–1427.
<https://doi.org/10.1007/s00180-015-0608-4>
- Asparouhov, T., & Muthén, B. (2024). Penalized structural equation models. *Structural Equation Modeling: A Multidisciplinary Journal*, 31(3), 429–454.
<https://doi.org/10.1080/10705511.2023.2263913>
- Blighe, K., Andrews, J., & Lun, A. (2026). *PCAtools: PCAtools: Everything principal components analysis*. <https://bioconductor.org/packages/PCAtools>
- Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2), 245–276. https://doi.org/10.1207/s15327906mbr0102_10

- Ding, C., & He, X. (2004). K-means clustering via principal component analysis. *Proceedings of the Twenty-First International Conference on Machine Learning*, 29.
- Epskamp, S., Cramer, A. O., Waldorp, L. J., Schmittmann, V. D., & Borsboom, D. (2012). Qgraph: Network visualizations of relationships in psychometric data. *Journal of Statistical Software*, 48, 1–18. <https://doi.org/10.18637/jss.v048.i04>
- Fan, J., Wang, K., Zhong, Y., & Zhu, Z. (2021). Robust high dimensional factor models with applications to statistical machine learning. *Statistical Science: A Review Journal of the Institute of Mathematical Statistics*, 36(2), 303. <https://doi.org/10.1214/20-sts785>
- Friedman, J. H., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22. <https://doi.org/10.18637/jss.v033.i01>
- Gajjar, S., Kulahci, M., & Palazoglu, A. (2017). Selection of non-zero loadings in sparse principal component analysis. *Chemometrics and Intelligent Laboratory Systems*, 162, 160–171. <https://doi.org/10.1016/j.chemolab.2017.01.018>
- Geminiani, E., Marra, G., & Moustaki, I. (2021a). *Penfa: Single- and multiple-group penalized factor analysis*.
- Geminiani, E., Marra, G., & Moustaki, I. (2021b). Single-and multiple-group penalized factor analysis: A trust-region algorithm approach with integrated automatic multiple tuning parameter selection. *Psychometrika*, 86, 65–95. <https://doi.org/10.1007/s11336-021-09751-8>
- Goretzko, D., & Bühner, M. (2020). One model to rule them all? Using machine learning algorithms to determine the number of factors in exploratory factor analysis. *Psychological Methods*, 25(6), 776. <https://doi.org/10.1037/met0000262>
- Gu, Z., Schipper, N. C. de, & Van Deun, K. (2019). Variable selection in the regularized simultaneous component analysis method for multi-source data integration. *Scientific Reports*, 9(1), 18608. <https://doi.org/10.1038/s41598-019-54673-2>
- Guerra-Urzola, R., Schipper, N. C. de, Tonne, A., Sijtsma, K., Vera, J. C., & Van Deun, K. (2023). Sparsifying the least-squares approach to PCA: Comparison of lasso and cardinality

- constraint. *Advances in Data Analysis and Classification*, 17(1), 269–286.
<https://doi.org/10.1007/s11634-022-00499-2>
- Horn, J. L. (1965). A rationale and test for the number of factors in factor analysis. *Psychometrika*, 30, 179–185. <https://doi.org/10.1007/BF02289447>
- Huang, P.-H. (2020). Lslx: Semi-confirmatory structural equation modeling via penalized likelihood. *Journal of Statistical Software*, 93(7), 1–37. <https://doi.org/10.18637/jss.v093.i07>
- Huang, P.-H., Chen, H., & Weng, L.-J. (2017). A penalized likelihood method for structural equation modeling. *Psychometrika*, 82(2), 329–354.
<https://doi.org/10.1007/s11336-017-9566-9>
- Kaiser, H. F. (1960). The application of electronic computers to factor analysis. *Educational and Psychological Measurement*, 20(1), 141–151. <https://doi.org/10.1177/001316446002000116>
- Le, T. T., Vermunt, J. K., Ballhausen, N., & Van Deun, K. (2026). Exploratory structural equation modeling and the curse of dimensionality. *Behavior Research Methods*, 58(3), 84.
<https://doi.org/10.3758/s13428-026-02960-y>
- Morin, A. J. S. (2014). Exploratory structural equation modeling. In R. H. Hoyle (Ed.), *Handbook of structural equation modeling* (pp. 503–524). Guilford Press.
- Nishimura, Y., Martin, C. L., Vazquez-Lopez, A., Spence, S. J., Alvarez-Retuerto, A. I., Sigman, M., Steindler, C., Pellegrini, S., Schanen, N. C., Warren, S. T., et al. (2007). Genome-wide expression profiling of lymphoblastoid cell lines distinguishes different forms of autism and reveals shared pathways. *Human Molecular Genetics*, 16(14), 1682–1698.
<https://doi.org/doi.org/10.1093/hmg/ddm116>
- O'Connor, B. P. (2026). *EFA.dimensions: Exploratory factor analysis functions for assessing dimensionality*. <https://doi.org/10.32614/CRAN.package.EFA.dimensions>
- Reise, S. P., Waller, N. G., & Comrey, A. L. (2000). Factor analysis and scale revision. *Psychological Assessment*, 12(3), 287. <https://doi.org/10.1037/1040-3590.12.3.287>
- Trendafilov, N. T. (2014). From simple structure to sparse components: A review. *Computational Statistics*, 29, 431–454.

Van Deun, K., Le, T., Malinowski, J., Mols, F., & Schoormans, D. (2025). *Regularized multigroup exploratory approximate factor analysis for easy analysis of complex data*. Center for Open Science.

William Revelle. (2026). *Psych: Procedures for psychological, psychometric, and personality research*. Northwestern University. <https://CRAN.R-project.org/package=psych>

Appendix A

Mathematical derivations for SEAFa algorithm

Here, we present the mathematical derivations for the update of factor scores and factor loadings for SEAFa.

1. Update of the factor scores conditional upon fixed values for the loadings is based on the following objective:

$$\begin{aligned} \min_{\eta_{11}, \dots, \eta_{iq}, \dots, \eta_{NQ}} \sum_{j=1}^J \sum_{i=1}^N \left(y_{ij} - \sum_q \eta_{iq} p_{jq} \right)^2 \\ \text{subject to } \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q. \end{aligned} \quad (\text{A1})$$

(A1) can be rewritten as the following equivalent problem:

$$\begin{aligned} \min_{\eta_1, \dots, \eta_q, \dots, \eta_Q} \left\| \mathbf{Y} - \sum_q \eta_q \mathbf{p}_q^T \right\|^2 \\ \text{subject to } \eta_q^T \eta_q = N \quad \forall q = 1, \dots, Q. \end{aligned} \quad (\text{A2})$$

Using the method of Lagrange Multipliers, we obtain the Lagrangian function

$$\begin{aligned} \mathcal{L} &= \left\| \mathbf{Y} - \sum_q \eta_q \mathbf{p}_q^T \right\|^2 - \mu (\eta_q^T \eta_q - N) \\ &= \left\| \mathbf{Y} - \sum_{t \neq q} \eta_t \mathbf{p}_t^T - \eta_q \mathbf{p}_q^T \right\|^2 - \mu (\eta_q^T \eta_q - N) \\ &= \left\| \mathbf{E}_q - \eta_q \mathbf{p}_q^T \right\|^2 - \mu (\eta_q^T \eta_q - N) \end{aligned} \quad (\text{A3})$$

Solving $\frac{\partial \mathcal{L}}{\partial \eta_q} = 0$ and $\frac{\partial \mathcal{L}}{\partial \mu} = 0$, we have $\eta_q = \frac{\sqrt{N} \mathbf{E}_q \mathbf{p}_q}{\sqrt{(\mathbf{E}_q \mathbf{p}_q)^T (\mathbf{E}_q \mathbf{p}_q)}}$.

2. The update of factor loadings conditional upon fixed values for the scores is based on the following objective:

$$\begin{aligned} \min_{\mathbf{P}} \left\| \mathbf{Y} - \mathbf{H} \mathbf{P}^T \right\|_F^2 \\ \text{subject to } \text{Card}(\mathbf{P}) = C, \end{aligned} \quad (\text{A4})$$

which can be rewritten in vectorized form as follows:

$$\min_{\mathbf{P}} \|\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H})\text{vec}(\mathbf{P}^T)\|_2^2, \quad (\text{A5})$$

subject to $\text{Card}(\mathbf{P}) = C$.

As pointed out by (Adachi & Kiers, 2017), (A5) can be majorized as follows:

$$\|\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H})\text{vec}(\mathbf{P}^T)\|_2^2 \leq c + \alpha \|\mathbf{b} - \text{vec}(\mathbf{P}^T)\|_2^2, \quad (\text{A6})$$

where the right-hand side function is also known as the surrogate function. Here, c is a constant with respect to $\mathbf{P}$, α is the maximum eigenvalue of $\mathbf{H}^T \mathbf{H}$. $\mathbf{b}$ is derived as follows:

$$\begin{aligned} \mathbf{b} &= \text{vec}(\mathbf{P}_{old}^T) - \frac{1}{\alpha} \left((\mathbf{I} \otimes \mathbf{H})^T (\mathbf{I} \otimes \mathbf{H}) \text{vec}(\mathbf{P}_{old}^T) - (\mathbf{I} \otimes \mathbf{H})^T \text{vec}(\mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}_{old}^T) - \frac{1}{\alpha} \left((\mathbf{I} \otimes \mathbf{H}^T) (\mathbf{I} \otimes \mathbf{H}) \text{vec}(\mathbf{P}_{old}^T) - (\mathbf{I} \otimes \mathbf{H}^T) \text{vec}(\mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}_{old}^T) - \frac{1}{\alpha} \left(\mathbf{I} \otimes (\mathbf{H}^T \mathbf{H}) \text{vec}(\mathbf{P}_{old}^T) - \text{vec}(\mathbf{H}^T \mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}_{old}^T) - \frac{1}{\alpha} \left(\text{vec}(\mathbf{H}^T \mathbf{H} \mathbf{P}_{old}^T) - \text{vec}(\mathbf{H}^T \mathbf{Y}) \right) \\ &= \text{vec} \left(\mathbf{P}_{old}^T - \frac{1}{\alpha} (\mathbf{H}^T \mathbf{H} \mathbf{P}_{old}^T - \mathbf{H}^T \mathbf{Y}) \right), \end{aligned}$$

or in matrix form as

$$\mathbf{B} = \mathbf{P}_{old} - \frac{1}{\alpha} (\mathbf{P}_{old} \mathbf{H}^T \mathbf{H} - \mathbf{Y}^T \mathbf{H}), \quad (\text{A7})$$

where $\mathbf{b} = \text{vec}(\mathbf{B}^T)$. Let $k = J \times Q - C$ be the number of zero loadings in $\mathbf{P}$ and $|b|_{(k)}$ is the k -th smallest element of $\mathbf{B}$. Then the update of $\mathbf{P} = (p_{jq})$ is

$$p_{jq} = \begin{cases} b_{jq}, & \text{if } |b_{jq}| \leq |b|_{(k)}, \\ 0, & \text{otherwise} \end{cases} \quad (\text{A8})$$

Appendix B

Demonstration of using the LASSO penalty in the seafar package

Instead of performing a model selection procedure to choose the number of (non)zero loadings, SEAFA with the LASSO penalty requires tuning the penalty parameter. This can also be done using the Index of Sparseness by running the following lines:

```
# estimate the maximum value for the LASSO penalty parameter
max_lambda <- lambda_max(
  data = big5_std,
  nfactors = 5,
  INIT = 'semirational'
)

# model selection using IS to choose the LASSO penalty parameters
sel_lasso <- is.seafar_lasso(
  data = big5_std,
  nfactors = 5,
  max_lambda = max_lambda,
  INIT = 'semirational',
  nstarts = 5
)
```

With the selected LASSO penalty parameter, the SEAFA model with the LASSO penalty can be fitted as follows:

```
# fit SEAFA with LASSO penalty
fit_lasso <- seafar_lasso_multistart(
  data = big5_std,
  nfactors = 5,
  lambda = sel_lasso,
  INIT = 'semirational',
  nstarts = 50
)

summary(fit_lasso)
```

Similarly, the `summary()` function can be used to quickly print the results. The full factor score and loading matrices can be extracted using `fit_lasso$scores` and `fit_lasso$loadings`, respectively.